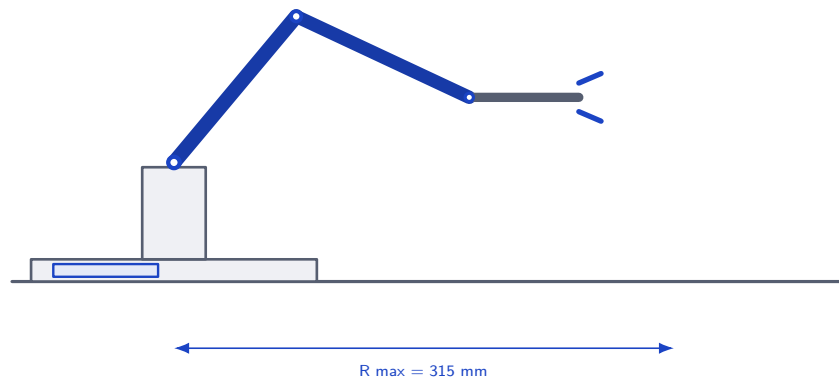


CAD \$50–70 target • \$100 ceiling • 3D-printed • SolidWorks

Desk Arm

A Low-Cost 3-DOF Desktop Robot Arm

V1 Design & Planning Guide



Configuration B — Recommended

315 mm reach • 55 g payload • shoulder safety factor 2.44 • ~CAD \$59

Contents

1	Executive summary	3
2	Requirements	3
2.1	Scope	3
2.2	Degrees of freedom	3
2.3	Target dimensions	3
2.4	Payload and workspace	4
2.5	Budget	4
2.6	Decision gate	4
3	Architecture	4
3.1	Why a slaved wrist rather than a parallelogram linkage	4
4	Geometry	5
5	Actuator selection	6
5.1	Method	6
5.2	Mass budget	6
5.3	Torque calculations	6
5.4	Servo table	7
5.5	Gripper: SG90 vs MG90S	7
6	Base stability	7
6.1	Fixes	8
7	Kinematics	8
7.1	Forward kinematics	8
7.2	Inverse kinematics	9
7.3	Validation order, before anything moves	9
8	Electronics	9
8.1	Controller: ESP32 (DevKitC or equivalent)	9
8.2	Serial protocol	10
9	Power system	10
10	Software architecture	11
11	Mechanical design and printing	11
11.1	Material assignment	11
11.2	Layer orientation — three rules that decide whether this works	12
11.3	Where FEA earns its time	12
11.4	Parametric strategy	12
12	Bill of materials	13
12.1	BOM A — purchase	13
12.2	BOM B — university-sourced	13
12.3	Cost summary	13
13	Configuration comparison	14
13.1	Decision trail	14

14 Component fallbacks	14
14.1 Geometry fallback — apply strictly in this order	15
15 Design risks	15
16 Build timeline	15
17 V1 → V2 roadmap	16
18 Decision gate	16

1 Executive summary

Build Configuration B. Direct-drive revolute arm, equal 120 mm links, 315 mm reach, 55 g design payload. MG996R at the shoulder and elbow, MG90S at the base and gripper, an SG90 slaved as a wrist leveller, ESP32 over USB serial. Worst-case shoulder safety factor is **2.44**. Purchase cost ~**CAD \$59** with typical university-lab sourcing (\$93 worst case, buying everything — still under the \$100 ceiling).

Two things the arithmetic changes about the design, not just describes:

- **The base tips over without ballast.** 224 g of moving arm on a 211 mm lever beats a 269 g base on a 90 mm lever (tipping safety factor 0.89). A 400 g ballast cavity in the base plate fixes it (safety factor 2.22) — see §6.
- **Do not thicken the printed links.** Tip deflection is 0.075 mm against ~6 mm of servo backlash — the structure is already about $80\times$ stiffer than the servos it is bolted to. Added material there costs shoulder margin for no measurable stiffness benefit — see §11.

Configuration A (ultra-budget, 275 mm reach, \$48–55) is viable but saves only ~\$8 over B for a materially smaller robot. Configuration D (stronger actuators, \$85–95) is not justified — every joint in Configuration B already clears the $SF \geq 2.0$ floor with margin. Full comparison in §13.

2 Requirements

2.1 Scope

V1 is a mechanically and electronically functional 3-DOF arm, controlled manually or programmatically from a PC over USB. No camera, no vision, no AI agent — that is V2 (§17), and V1’s software is only required not to block it.

2.2 Degrees of freedom

1. **Base yaw** — rotates the whole arm about a vertical axis.
2. **Shoulder** — raises/lowers the upper arm in the vertical plane the base currently points at.
3. **Elbow** — folds the forearm relative to the upper arm, same plane.
4. **Gripper** (not a positioning DOF) — opens/closes a two-finger claw.
5. **Wrist pitch** — present in hardware (an SG90) but *not* an independently commanded axis. Firmware slaves it to $-(\theta_1 + \theta_2)$ so the gripper stays level in every pose (§7).

This is a positioner with 3 controllable spatial DOF (x, y, z) plus grip — matching the brief exactly, no more.

2.3 Target dimensions

Parameter	Target (brief)	Chosen
Base footprint	150–200 mm	180×180 mm
Reach	250–350 mm	315 mm kinematic / 290 mm practical
Overall height	250–400 mm	~340 mm (arm vertical, incl. gripper)

Link lengths: shoulder height $h_0 = 75$ mm, upper arm $L_1 = 120$ mm, forearm $L_2 = 120$ mm, wrist→TCP $L_3 = 75$ mm. Equal $L_1 = L_2$ is deliberate — see §4.

2.4 Payload and workspace

Design payload: **55 g** at full 315 mm reach — pens, small tools, USB sticks, small project boxes, small electronics.

Bound	Value	Set by
Outer radius	315 / 290 mm	Link length / IK conditioning near full extension
Inner radius	130 mm	Forearm-to-turret collision
Floor	$z = 10$ mm	Jaw-to-table clearance, enforced in firmware
Ceiling	$z = 315$ mm	Arm vertical
Base sweep	$\pm 90^\circ$	No slip ring in V1

Usable desk area: an annular sector, 130–290 mm radius, 180° — about 0.09 m^2 directly in front of the base.

2.5 Budget

Target personal spend **CAD \$50–70**; absolute maximum **CAD \$100**. Configuration B lands at **~\$59** with typical lab sourcing, **~\$93** buying everything. Full breakdown in §12.

2.6 Decision gate

Four questions, answered before any implementation started:

1. **Configuration B**, built up from A rather than starting at D.
2. **SG90 slaved wrist: in.** \$3 and 17 g buys a level gripper in every pose.
3. **University sourcing:** assume lab-available for controller, bench power supply, fasteners, wire, bearings, filament; budget the MG996R pair and MG90S/SG90 as purchased.
4. **315 mm / 55 g is the right target** — no known heavier payload requirement.

3 Architecture

Serial revolute arm — the classic RRR shoulder–elbow chain on a rotating turret. Base yaw is the only vertical axis; shoulder and elbow work in a single vertical plane the base sweeps around. This is the simplest structure that reaches every point on a desk, and the kinematics collapse to one trig problem.

Axis	Type	Range	Actuator	Notes
1 — Base yaw	Revolute, vertical	$-90^\circ \dots +90^\circ$	MG90S	Turret on 2×608ZZ bearings
2 — Shoulder	Revolute, horiz.	$0^\circ \dots 110^\circ$	MG996R	Highest-loaded joint
3 — Elbow	Revolute, horiz.	$-130^\circ \dots 0^\circ$	MG996R	Elbow-up solution only
Wrist (slaved)	Revolute, horiz.	$-120^\circ \dots +30^\circ$	SG90	Firmware-computed, not commanded
Gripper	Two-finger, meshed	0–55 mm	MG90S	Compliant tip

3.1 Why a slaved wrist rather than a parallelogram linkage

A three-DOF positioner has exactly enough freedom to hit an (x, y, z) target and none left over — gripper pitch is whatever the IK happens to produce, swinging roughly 30° across the workspace.

Three ways to fix that:

Twin parallelogram

Two four-bar linkages hold the jaws level with no extra servo, and move the elbow actuator to the turret. Elegant, but six more printed parts and twelve more pivots with slop at every one. *Deferred.*

SG90 slaved wrist — chosen

One extra micro servo, driven by firmware as a dependent axis. Costs ~\$3 and 17 g at 250 mm, which drops shoulder SF from 2.57 to 2.44 — cheap for level jaws.

Fixed bracket

Bolt the gripper on at a fixed 45°. Zero cost, zero parts, works if objects are always approached from the side. *Configuration A fallback.*

All three bolt to the same 24×24 mm wrist mounting face on the forearm — start with the fixed bracket, add the SG90 later, try the linkage after that, without touching another part in the assembly.

4 Geometry

Equal-length links, $L_1 = L_2 = 120$ mm — deliberate, not arbitrary: equal lengths make the printed parts nearly identical (one parametric sketch, two configurations), maximise the folded-to-extended reach ratio, and put the workspace’s inner boundary where the turret already is. Every dimension traces back to the torque budget in §5.

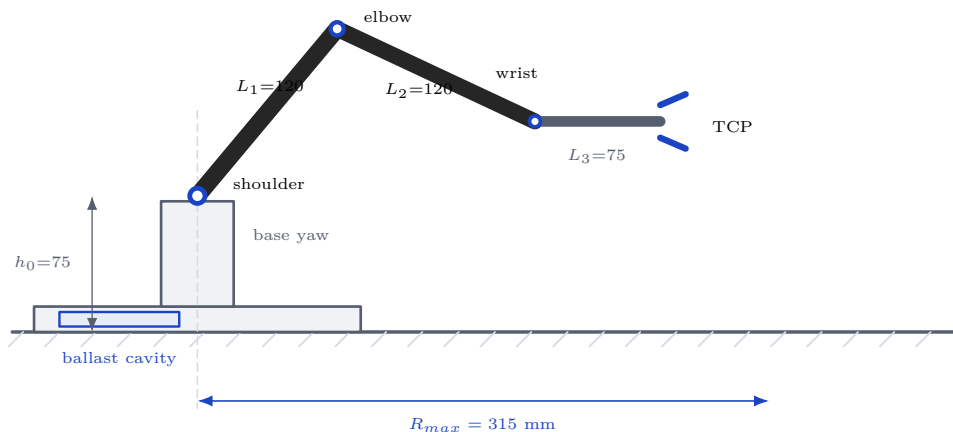


Figure 1: Side elevation, 1 px = 1 mm, arm shown mid-stroke. The wrist stays level in every pose, so L_3 is a purely horizontal offset in the kinematics.

Parameter	Value	Rationale
Shoulder height h_0	75 mm	Elbow folds without hitting the turret
Upper arm L_1	120 mm	Torque-driven — 140 mm drops shoulder SF below 2.1
Forearm L_2	120 mm	Equal to L_1 — same printed profile
Wrist→TCP L_3	75 mm	Jaw depth + clearance for a 30 mm object
Max TCP reach	315 mm	Within the 250–350 mm brief
Base footprint	180×180 mm	Drives the tipping calculation, §6
Jaw opening	0–55 mm	Pen through small project box

5 Actuator selection

5.1 Method

Static gravity-holding torque at the worst-case pose (fully extended, horizontal — also the normal working pose for a desk arm reaching across the table). Dynamic terms are small at the speeds this arm moves at (under 10% of the gravity term at the firmware’s 60°/s slew limit) and are absorbed by the safety-factor floor rather than modelled explicitly. Target safety factor: ≥ 2.0 , evaluated at 6.0 V rated torque — roughly the point below which a hobby servo holding a static pose runs hot instead of idling near stall current.

5.2 Mass budget

Horizontal distance from the shoulder axis, arm fully extended — the worst case for every load in the chain. Verified by `calculations/engineering_calcs.py`.

Item	Mass	Distance	Moment
Upper-arm link pair + spacers	30 g	60 mm	1,800 g · mm
Elbow servo (MG996R) + bracket	65 g	120 mm	7,800 g · mm
Forearm link pair + spacers	25 g	180 mm	4,500 g · mm
Wrist servo (SG90) + bracket	17 g	250 mm	4,250 g · mm
Gripper assembly (MG90S + jaws)	32 g	285 mm	9,120 g · mm
Design payload	55 g	320 mm	17,600 g · mm
Moving assembly total	224 g	CoM 201 mm	45,070 g · mm

Below the shoulder (drives base stability, not shoulder torque): base plate 100 g, column + turret 60 g, base servo 14 g, shoulder servo 55 g, hardware 40 g → **269 g**.

The elbow servo alone is 29% of the moving mass, sitting at the point in the chain where moving it costs the most shoulder torque — the highest-leverage future change, if shoulder margin ever becomes a problem, is relocating it to the turret behind a linkage (~1.4 kg · cm), not implemented in V1.

5.3 Torque calculations

$$\text{Shoulder: } \tau_1 = g \sum_i m_i d_i = 9.81 \times 0.04507 \text{ kg m} = 0.442 \text{ N m} = 4.51 \text{ kg} \cdot \text{cm}$$

$$\begin{aligned} \text{Elbow: } \tau_2 &= 9.81 \times (0.025 \times 0.060 + 0.017 \times 0.130 + 0.032 \times 0.165 + 0.055 \times 0.200) \\ &= 9.81 \times 0.01999 = 0.196 \text{ N m} = 2.00 \text{ kg} \cdot \text{cm} \end{aligned}$$

$$\text{Base yaw: } \tau_0 \approx I\alpha + \text{drag} = 0.0056 \times 3.5 + \text{drag} \approx 0.30 \text{ kg} \cdot \text{cm}$$

$$\text{Wrist: } \tau_3 = g [0.032(0.285 - 0.240) + 0.055(0.320 - 0.240)] = 0.057 \text{ N m} = 0.58 \text{ kg} \cdot \text{cm}$$

5.4 Servo table

Joint	Servo	Req.	Target $\times 2$	Rated @6V	SF
Base	MG90S	0.30	0.60	2.20	7.34 OK
Shoulder	MG996R (<i>sizing joint</i>)	4.51	9.02	11.00	2.44 OK
Elbow	MG996R	2.00	4.00	11.00	5.50 OK
Wrist (slave)	SG90	0.58	1.17	1.80	3.08 OK
Gripper	MG90S	0.82	1.64	2.20	2.70 OK

All $\text{kg} \cdot \text{cm}$. On a 5 V supply the MG996R drops to $\sim 9.4 \text{ kg} \cdot \text{cm}$ and shoulder SF falls to 2.08 — still acceptable, which is why a 5 V/3 A phone charger is a legitimate power fallback (§9).

Why the elbow is over-specified (SF 5.50), deliberately: it needs $2.00 \text{ kg} \cdot \text{cm}$, so at SF 2 it needs 4.0. The MG90S tops out at 2.2 and fails. The next rung up, an MG92B at $3.5 \text{ kg} \cdot \text{cm}$ ($\sim \$8$), still gives only SF 1.75 — and costs more than an MG996R. The MG996R wins by default and the margin is free.

5.5 Gripper: SG90 vs MG90S

Required clamp force for a 55 g object, $\mu = 0.5$ (rubber pad, conservative): $mg/2\mu = 0.54 \text{ N}$. Design target: **4 N per jaw** — about $7\times$ the physics minimum, because real grips are off-centre and objects get bumped.

$$\text{Jaw pivot torque} = 4.0 \text{ N} \times 0.030 \text{ m} = 0.12 \text{ N m}$$

$$\text{Servo torque (1.5:1 reduction)} = 0.12/1.5 = 0.080 \text{ N m} = 0.82 \text{ kg} \cdot \text{cm}$$

	SG90	MG90S (chosen)
Mass / cost	9 g / $\sim \$2$	13.4 g / $\sim \$6$
Gears	Nylon — strips on a hard stall	Metal — survives stalls/knocks
Footprint	$22.8 \times 12.2 \times 22.5 \text{ mm}$, 32 mm span	Identical — drop-in either way

Both pass on torque. The decision is about what happens the twentieth time the gripper closes on something it cannot close on — metal gears survive it, nylon eventually strips. The identical footprint means the mount is one printed part regardless of which servo goes in it.

Mechanism: two mirror-image printed jaws on M3 pins with meshed gear sectors (module 1.5, ~ 10 teeth engaged) — one servo drives one jaw, the sectors force the other to mirror it. Self-centring, cannot desync. Jaw faces carry a 90° V-groove plus a rubber pad. **Compliance matters more than gear material:** thin the last 12 mm of each jaw to 1.6 mm so it flexes $\sim 1 \text{ mm}$ under load — one commanded “closed” angle then grips a range of object sizes without ever driving the servo into a hard stall.

6 Base stability

Tipping about the front edge of the base plate, arm fully extended, payload at maximum reach. The shoulder axis sits 10 mm forward of the base axis, so the moving CoM is 211 mm out.

$$M_{\text{over}} = 0.224 \times 9.81 \times (0.211 - 0.090) = 0.266 \text{ N m}$$

$$M_{\text{rest, no ballast}} = 0.269 \times 9.81 \times 0.090 = 0.238 \text{ N m} \quad \Rightarrow \quad \text{SF} = 0.89 \quad \text{fails}$$

$$M_{\text{rest, +400 g ballast}} = 0.669 \times 9.81 \times 0.090 = 0.591 \text{ N m} \quad \Rightarrow \quad \text{SF} = 2.22 \quad \text{OK}$$

The arm as drawn tips over. 224 g on a 211 mm lever beats a 269 g base on a 90 mm lever, and no amount of careful printing fixes that — it is easy to miss on paper because the whole robot only weighs ~500 g.

6.1 Fixes

Ballast the base — primary

Seal a 66×66×18 mm cavity into the base plate with a snap lid. Fill with washers, sand, or spare screws. **400 g minimum, 500 g preferred.** Costs nothing and lowers the CoM as well as raising the mass.

Park the supply on the plate

A 6 V 3 A brick weighs 150–250 g. Recessed behind the column, it becomes free ballast plus shorter power leads.

Clamp it down — optional

Two 8 mm slots at the rear take a small C-clamp. Makes tipping structurally impossible, worth having for fast moves, but should not be the only thing keeping the arm upright.

Dynamic loading is worse than this static calculation. Decelerating the arm at the end of a fast reach adds an inertial term this analysis ignores — the target is SF 2.2, not 1.2, and firmware slew-limits every joint to 60°/s for exactly this reason.

7 Kinematics

No DH tables, no Jacobians, no numerical solver. Base rotation decouples completely from shoulder/elbow, leaving a two-link planar problem with a closed-form solution — implemented in `software/robot/kinematics.py` (~40 lines including workspace checks, with a self-test that reproduces every number in this section).

7.1 Forward kinematics

$$r = L_1 \cos \theta_1 + L_2 \cos(\theta_1 + \theta_2) + L_3$$

$$z = h_0 + L_1 \sin \theta_1 + L_2 \sin(\theta_1 + \theta_2)$$

$$x = r \cos \theta_0 \quad y = r \sin \theta_0$$

L_3 adds directly to r with no trig term — the payoff of the levelled wrist. Without it, L_3 would carry a $\cos(\theta_1 + \theta_2 + \theta_3)$ factor and the inverse problem stops being closed-form.

7.2 Inverse kinematics

$$\theta_0 = \text{atan2}(y, x)$$

$$r' = \sqrt{x^2 + y^2} - L_3 \quad z' = z - h_0$$

$$D = \frac{r'^2 + z'^2 - L_1^2 - L_2^2}{2L_1L_2} \quad (\text{unreachable if } |D| > 1, \text{ checked before acos})$$

$$\theta_2 = -\arccos(D) \quad (\text{negative root} = \text{elbow-up})$$

$$\theta_1 = \text{atan2}(z', r') - \text{atan2}(L_2 \sin \theta_2, L_1 + L_2 \cos \theta_2)$$

$$\theta_3 = -(\theta_1 + \theta_2) \quad (\text{slaved wrist, jaws level})$$

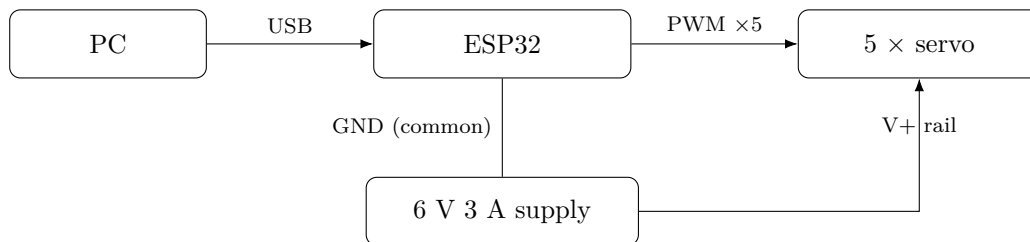
The two arccos roots are elbow-up and elbow-down. Elbow-down puts the elbow below the table on nearly every desk-height target, so the solver takes the negative root unconditionally.

7.3 Validation order, before anything moves

1. **Reachability** — $|D| \leq 1$ and $130 \leq \sqrt{x^2 + y^2} \leq 290$ mm.
2. **Joint limits** — every θ inside its mechanical range (§3.2).
3. **Table clearance** — $z \geq 10$ mm, and the elbow itself above the table.
4. **Path check** — for a Cartesian move, interpolate 20 waypoints and validate each; a straight line between two legal poses can pass through an illegal one.

The accuracy limit is backlash, not mathematics. An MG996R has 1–2° of gear lash plus potentiometer deadband. At the 240 mm shoulder+elbow lever, 1.5° is **6 mm at the gripper**. Expect ± 5 –**8 mm** repeatability and design around it: approach every target from the same direction, and let the 55 mm jaw opening absorb the rest. No amount of IK precision buys back a millimetre of this.

8 Electronics



8.1 Controller: ESP32 (DevKitC or equivalent)

Full pin-out, colour-coded nets and the grounding/power checklist below are also drawn out as a standalone diagram, `wiring_diagram.svg`, alongside this guide. Revised from an original Arduino Nano choice. The Nano’s advantage was native **5 V logic** — hobby servos want a 5 V pulse, and a 3.3 V board introduces a question with no clean answer on paper: some servos latch reliably on 3.3 V, some jitter, and the difference only shows up after assembly. In practice, the SG90/MG90S/MG996R family used here decodes PWM through an analog RC input stage that a 3.3 V high clears — a well-worn combination, not a gamble. **Test each servo before final assembly**; the documented fallback for a unit that jitters is a 74HCT125/74AHCT125 quad buffer (~\$1, covers all five signal lines), not a controller change. In exchange for accepting that one check: a dual-core chip with far more flash/RAM/GPIO than this project needs, a similar street price to the Nano, and Wi-Fi/BT already on board if a future V2 ever wants the arm untethered (unused,

not required, by docs/v2_ai_roadmap.md). Requires the ESP32Servo library in place of the stock Servo.h (the ESP32 Arduino core ships neither) — the attach()/writeMicroseconds() API is unchanged, it just rides the chip’s LEDC PWM peripheral instead of a hardware timer.

Deliberately not included: a PCA9685 driver board (five servos fit on five ESP32 GPIOs already — the board solves a problem that starts around twelve servos); encoders or current sensing (hobby servos have no position-feedback path to expose); limit switches (servos are absolute-position devices and already know where they are at power-on).

Fallback: an Arduino Nano/Uno remains a straightforward substitute if an ESP32 isn’t available, or a servo won’t settle at 3.3 V even behind a level shifter — same pin roles, same serial protocol, swap ESP32Servo back for stock Servo.h and drop the timer-allocation calls in setup().

8.2 Serial protocol

ASCII, line-oriented, one command per line, every command answered.

Command	Reply	Purpose
PING	OK ARM v1	Port identification and liveness
J <base> <sh> <el>	OK / ERR LIMIT <joint>	Absolute joint angles, degrees
G <0–100>	OK	Gripper, 0 closed, 100 open
US <ch> <us>	OK / ERR RANGE	Raw pulse, ch=B/S/E/W/G, calibration only
HOME	OK	Slew to the safe folded pose
STOP	OK	Freeze at the current position
REL	OK	Detach all servos
STAT	angles + us	Current commanded state

The split is deliberate: **the firmware owns safety, the PC owns intelligence**. Joint limits and slew limiting live on the ESP32 and cannot be bypassed by a buggy Python script — or, later, a confused AI agent.

9 Power system

Servo	Qty	Idle	Moving	Stall
MG996R	2	0.17 A	0.6–0.9 A	2.5 A
MG90S	2	0.08 A	0.2–0.4 A	0.7 A
SG90	1	0.06 A	0.2 A	0.65 A
Rail total	5	0.56 A	1.2–1.8 A	7.05 A

The 7.05 A all-stall figure is not a design case — it requires every joint jammed simultaneously. The design case is **1.8 A continuous**, **~3.5 A transient** at direction reversals.

Specification: 6.0 V, 3 A regulated (18 W), barrel jack, 3 A fuse. 6.0 V because MG996R torque is quoted at 6 V; at 5 V shoulder SF falls from 2.44 to 2.08. Bulk capacitance: $2 \times 1000 \mu\text{F}$ at the servo headers. $C\Delta V/I = 2000 \mu\text{F} \times 0.5\text{V}/2\text{A} = 0.5 \text{ ms}$ — enough to kill a switching spike, *not* a substitute for a supply that can source the current.

Three rules, each of which has ended someone’s project: never feed the servo rail from the ESP32’s 5V/VIN pin or USB (that pin sits upstream of the board’s own 3.3 V logic regulator

and reflects whatever the USB port can source, typically ~500 mA; five servos will brown it out mid-move); servo ground and ESP32 ground must be joined, or the PWM has no reference and servos twitch randomly; no 7.4 V LiPo direct to the rail — it will destroy the MG90S and SG90 within minutes.

10 Software architecture

Layer	Owns
firmware/arm_firmware/ (Arduino C++)	Joint limits, slew rate — <i>safety</i>
software/robot/ (Python)	Kinematics, trajectory, transport, the Arm API
software/cli.py, sequences.py	Manual control and demo/test routines
V2 (<i>not built</i>)	software/vision/, software/agent/ — add here only

The public surface everything else is meant to use — the CLI, the demo sequences, and eventually the V2 agent — is `robot.arm.Arm`:

```
arm.home()
arm.move_joints(base, shoulder, elbow)
arm.move_to(x, y, z)  (Cartesian, validated, interpolated)
arm.grip(percent)
arm.pickup(x, y, z)
arm.place(x, y, z)
```

This is the V1→V2 contract (§17): in V2 the agent does not get its own motion code, its own limits, or its own IK — it calls exactly these six methods. No ROS, no behaviour trees, no message bus — a 300 mm desk arm with five servos does not have a software problem that a Python class cannot solve.

11 Mechanical design and printing

Links are twin printed side plates on printed spacers — a box section that is stiff in bending, prints flat with no supports, and puts every bolt in double shear. Roughly 14 unique parts, 20 printed pieces, ~300 g of filament, 18–22 h print time.

11.1 Material assignment

Part	Material	Why
Base, column, turret	PLA	Stiffness and flatness; not impact-loaded
Upper arm, forearm links	PLA	$E \approx 3.5$ GPa vs. PETG's 2.0 — deflection matters most
Shoulder / elbow brackets	PETG	Highest bending stress; layer adhesion matters most
Servo horn adapters	PETG	Small, bolted, loaded to peel layers apart
Gripper jaws	PETG	Designed to flex; PLA would snap at the flex section
Spacers, clips, tray	PLA	Unloaded

Print settings: 0.4 mm nozzle, 0.2 mm layers, **4 perimeters**, 30% gyroid, 5 top/bottom — perimeter count dominates strength in a thin printed part.

11.2 Layer orientation — three rules that decide whether this works

1. **Print brackets flat, never standing up.** Loaded perpendicular to the layer planes, a printed part manages only ~40–60% of its in-plane strength.
2. **Every bolt hole axis is normal to the build plate.** Bolts then clamp *across* layers in compression, not trying to split them apart.
3. **Never print a servo spline.** A printed 25-tooth spline *will* round off. Bolt the metal horn that shipped with the servo to the printed part with 2–3 M2 screws — the single most common failure mode in printed arms, designed out from part one.

11.3 Where FEA earns its time

Worth running (2 studies): the shoulder bracket (0.44 N m through a bolted interface, stress concentration at the servo cutout — the one part where intuition is unreliable); the upper-arm link’s elbow-servo pocket (expect the answer to be “add a 3 mm fillet”). **Not worth running:** everything else — the hand calculation below already answers the question.

$$I = 2 \times \frac{3 \times 20^3}{12} = 4000 \text{ mm}^4$$

$$\delta = \frac{FL^3}{3EI} = \frac{1.3 \times 120^3}{3 \times 2500 \times 4000} = 0.075 \text{ mm}$$

Seventy-five microns. The structure is ~80× stiffer than the servo backlash it is bolted to (§7). **Do not thicken these links** — every gram added buys nothing measurable and costs shoulder safety factor directly.

11.4 Parametric strategy

One global variables file drives L_1 , L_2 , L_3 , h_0 , plate thickness and pin diameter across every part. A servo-pocket library feature with a two-row design table (standard: MG996R-class, 40.7×19.7×42.9 mm; micro: MG90S/SG90-class, 22.8×12.2×22.5 mm) means swapping servo sizes is a configuration change, not a redesign — this is what makes Configurations A–D one robot instead of four. 0.2 mm nominal clearance on every pin and pocket, tuned once via a printed tolerance coupon. Full part list and file layout in `cad/README.md`.

12 Bill of materials

12.1 BOM A — purchase

Item	Qty	Cost	Likely in a lab?
MG996R servo	2	\$17	Unlikely
MG90S servo	2	\$13	Maybe
SG90 servo	1	\$3	Likely
6 V 3 A adaptor + barrel jack	1	\$14	Likely (bench PSU substitutes)
ESP32 DevKit (CP2102/CH340)	1	\$12	Maybe — a Nano works as a \$9 fallback
1000 μ F 16 V electrolytic $\times 2$	1	\$3	Very likely
608ZZ bearing $\times 2$	1	\$4	Likely
M3 screw/nyloc assortment	1 kit	\$11	Very likely
Dupont wire + perfboard	1	\$8	Very likely
PLA + PETG filament (~300 g)	—	\$8	Likely
Buy absolutely everything		\$93	<i>under the \$100 ceiling</i>

Optional, not in the total above: a 74HCT125/74AHCT125 level shifter (~\$1) if a servo jitters on the ESP32's 3.3 V signal (§9) — most builds won't need it.

12.2 BOM B — university-sourced

Component	Saves	Substitution risk
Microcontroller (ESP32, or Uno/Nano/Mega/Pico fallback)	\$12	Low — confirm servo signal quality
Power supply (bench PSU)	\$14	None — better for testing
Fasteners (workshop M3 stock)	\$11	None
Wiring, headers, perfboard	\$8	None
Bearings (mechanism-lab stock)	\$4	Low
Filament (makerspace stock)	\$8	None
Capacitors	\$3	None
Micro servos (teaching kits)	\$16	Check for stripped gears first
MG996R $\times 2$	\$17	Plan to buy — unlikely on the shelf

12.3 Cost summary

Scenario	Cost
Buy everything (worst case)	\$93
Typical student (servos + supply + filament, rest sourced)	\$59
Well-stocked lab (MG996R + MG90S only)	\$33
AliExpress route (buy-everything, 3-week wait)	\$42

The \$59 typical-student figure is unaffected by the ESP32 swap — the controller is lab-sourced in that scenario either way. The AliExpress total moves by only \$1 despite the \$3 Amazon.ca delta, since ESP32 dev boards are inexpensive there too, often \$5–6.

13 Configuration comparison

	A — Ultra-budget	B — Recommended	C — University	D — Stronger
Purchase cost	\$48–55	\$55–65	\$17–35	\$85–95
Reach	275 mm	315 mm	315 mm	315 mm
Payload	30 g	55 g	55 g	80 g
Base	SG90	MG90S	lab micro servo	MG90S
Shoulder	MG996R	MG996R	MG996R	DS3218 (20 kg · cm)
Elbow	MG90S	MG996R	MG996R	MG996R
Gripper	SG90	MG90S	MG90S	MG90S
Wrist	fixed bracket	SG90 slaved	SG90 slaved	MG90S slaved
Shoulder SF	4.1	2.44	2.44	4.0
University components	minimal	minimal	high	optional

Every column is the *same robot*: same CAD files, same assembly, same firmware, same Python. A→B is a 40 mm forearm configuration change and two servo swaps. B→D is one servo and a different mounting plate.

13.1 Decision trail

G1 — Can the cheapest configuration meet torque requirements?

Yes, but only at 275 mm reach and 30 g payload — satisfies torque by shrinking the robot to the low end of the brief’s range. *Conditional pass.*

G2 — Can university components close the gap at zero cost?

Partly — ~\$45 of the BOM is plausible lab stock, but MG996R-class servos usually are not. *Partial.*

G3 — Is the step from A to B worth its cost?

Yes, decisively: \$8 buys +40 mm reach, +25 g payload, and a levelled wrist. **Configuration B selected.**

G4 — Does B leave enough margin, or is D required?

Every joint clears SF 2.0. A DS3218 would add \$18 to fix a problem that does not exist. **Configuration D not required.**

14 Component fallbacks

Component	Primary	If unavailable	If too weak	University
Shoulder	MG996R	MG995	DS3218	Any 9–20 kg · cm servo
Elbow	MG996R	MG90S + $L_2=80\text{mm}$	2nd MG996R	Standard-frame servo
Base	MG90S	SG90	MG996R	Any micro servo
Gripper	MG90S	SG90 + lighter jaws	MG92B	Check for stripped gears
Wrist	SG90	fixed 45° bracket	MG90S	Any micro servo
Controller	ESP32	Nano/Uno (5V)	—	ESP32/Uno/Nano/Pico
Power	6V 3A adapter	5V 3A charger (–15%)	6V 5A adapter	Bench supply

14.1 Geometry fallback — apply strictly in this order

1. **Cut the payload** — 55 g→30 g removes 39% of the shoulder moment for free.
2. **Shorten the forearm** — L_2 120→80 mm, reach falls to 275 mm.
3. **Lighten the prints** — 3 mm→2.5 mm plates, 30%→20% infill; ~12 g saved, deflection stays under 0.15 mm.
4. **Only then, a bigger servo** — and re-check steps 1–3 first.

A 350 mm arm at 55 g needs ~5.4 kg · cm at the shoulder — an MG996R right at the edge, or \$18 more for a DS3218. The 315 mm arm clears 4.51 comfortably on the \$8 servo. 35 mm of reach is not worth doubling the actuator budget.

15 Design risks

Risk	Severity	Mitigation
Printed servo-horn interface strips	fails	Never print splines — bolt the metal horn on. Designed in from part one.
Base tips forward	fails	400 g ballast cavity + supply on the plate (§6). Verify by hand before power-on.
Brownout resets mid-move	fails	Separate 3 A servo rail, common ground, 2×1000 µF, never the ESP32's 5V/VIN pin.
Servo jitters on ESP32's 3.3 V signal	watch	Test each servo before final assembly; add a 74HCT125 level shifter (~\$1) on that signal line, or fall back to a Nano (§12) — see §Controller in Electronics.
Backlash limits accuracy to ±6 mm	watch	Unfixable in software — approach from one direction, rely on jaw opening.
Gripper servo cooks while holding	watch	Compliant jaw tips + REL to detach after a grip is established.
Arm driven into the table	watch	Firmware clamps independently of the PC; software validates every waypoint.
Print tolerances bind the joints	watch	0.2 mm nominal clearance; print a tolerance coupon first.
Servo buzz at rest	OK	Detach on idle via REL — cosmetic.
PLA creep under sustained load	OK	Deflection is 0.075 mm at 5% of yield — don't park extended for days.

16 Build timeline

Assumes 6–10 h/week alongside coursework.

Week	Phase	Output	Done when
0	Requirements	This guide, signed off	Configuration chosen, order placed
1–2	Mechanical design	SolidWorks parts + assembly	Assembly rebuilds cleanly when L_1 changes
2–3	Print	~20 parts, 300 g	Everything fits without a file; budget 2 revisions
3	Assembly wiring	+ Arm on the desk	Every joint moves freely by hand
4	Firmware	Serial protocol, limits, slew	Each joint sweeps its range without binding
5	Kinematics calibration	+ Python layer, <code>arm.py</code>	<code>move_to()</code> lands within ± 8 mm
6	Testing	Payload, repeatability, thermal	Picks and places a pen 10 times running
7	Documentation	README, docs/, CAD exported	Someone else could rebuild it from the repo

6–8 weeks part-time. Phase 5 calibration is where schedules slip — per-servo microsecond offsets take a patient evening.

17 V1 → V2 roadmap

Nothing here gets built now — it is recorded so V1’s decisions do not close a door V2 needs open.

Stage	Adds	Cost	Touches V1?
2.1 Eyes	USB webcam on a printed overhead mast	\$0–15	One new printed part
2.2 Desk-plane calibration	Four-point homography: pixels → desk coords	\$0	New module — the whole coordinate problem, since the desk is flat
2.3 Detection	YOLO / colour-contour detector → centroids	\$0	New module
2.4 Agent	LLM with <code>arm.pickup()/arm.place()</code> as tools	\$0	Nothing — same six calls the CLI uses

Three decisions V1 makes on V2’s behalf, each free now and expensive to retrofit: the PC holds intelligence and the MCU holds safety, so an agent physically cannot exceed a joint limit; the arm’s public API is Cartesian, so vision output plugs straight in; the base plate has camera-mast mounting provision, so the camera-to-robot transform calibrates once and stays valid.

“Pick up my screwdriver and put it beside my laptop” resolves to: detect → centroid → homography → `arm.pickup(x,y,z)` → agent picks a target → `arm.place(x',y',z')`. Every step after the homography is V1 code that already works.

18 Decision gate

Four answers, recorded, that unlock implementation:

1. **Configuration B** — built up from A rather than starting at D.
2. **SG90 slaved wrist: in** — \$3 and 17 g for level jaws in every pose.
3. **University sourcing assumed** for controller, bench supply, fasteners, wire, bearings, filament; MG996R pair and MG90S/SG90 budgeted as purchases.

4. **315 mm / 55 g** is the design target — no known heavier payload requirement.

Every figure in this guide is reproduced by `calculations/engineering_calcs.py` (self-checking against the values above) and `software/robot/kinematics.py` (FK/IK round-trip self-test). Servo ratings are published manufacturer specs for generic parts — verify against what you actually receive, since MG996R quality varies between sellers.